Запрос API: Родион (Clarification Engine)

Родион, оркестратор вызывает твой движок двумя методами: **`assess`** решает, хватает ли данных и какие вопросы задать, **`parse_answers`** разбирает ответ пользователя. Движок ничего не пишет в БД и не говорит с пользователем: он возвращает решение, а исполняет его оркестратор. Модели данных — в разделе 2 вкладки «Оркестрация».

1. Когда оркестратор тебя вызывает

Ситуация	Метод	`trigger`	Что уже есть на входе
Новая задача, контекст собран	`assess`	`initial`	запрос, контекст, проекты чата
Пользователь ответил, ответы сохранены	`assess`	`after_answer`	+ `asked`, `answers`; слоты с `source=USER` уже в `task.frame`
Модель во время выполнения вызвала `request_clarification`	`assess`	`execution_uncertainty`	+ `uncertainty` — текст модели о том, чего не хватает
Пользователь написал, пока задача в `waiting_user`	`parse_answers`	—	открытые вопросы, текст сообщения, проекты чата

2. Интерфейс

```
class ClarificationEngine(Protocol):
    async def assess(self, inp: AssessmentInput) -> Assessment
    async def parse_answers(self, inp: AnswerInput) -> AnswerParse

# ---- assess
class AssessmentInput:
    task: Task                    # request_text, frame (уже известные
слоты), project_id
    context: ContextBundle        # отобранные блоки контекста
    history: list[Message]        # все сообщения задачи
    asked: list[Question]         # все вопросы, заданные раньше (со
статусами)
    answers: list[Answer]
    projects: list[Project]       # кандидаты для слота project_id
    trigger: initial | after_answer | execution_uncertainty
    uncertainty: str | None
    round: int                    # номер раунда, если вернёшь ASK
    max_questions: int            # K = 3
    rounds_left: int              # 0 = спрашивать больше нельзя

class Assessment:
    decision: ASK | PROCEED | ABORT
    goal: str | None
    resolved_slots: list[Slot]    # найдено в контексте, source=CONTEXT
    gaps: list[Gap]               # все найденные пробелы
    questions: list[Question]     # отранжированы по priority, только при ASK
    assumptions: list[Assumption] # только для НЕблокирующих пробелов
    reason: str                   # при ABORT покажу пользователю
    usage: Usage                  # токены твоих вызовов LLM

class Gap:        id; slot_name; description; blocking: bool
                  information_gain; impact; user_cost; available_in_context
# 0..1
class Question:   id; task_id; gap_id; slot_name; text; options: list[str];
priority; round
class Slot:       name; value; source: USER | TOOL | CONTEXT | ASSUMPTION;
evidence: list[str]; confidence
class Assumption: slot_name; value; rationale

# ---- parse_answers
class AnswerInput:  task: Task; pending: list[Question]; message: Message;
projects: list[Project]
class AnswerParse:
    intent: ANSWER | NEW_REQUEST | CANCEL | UNCLEAR
```

```
    answers: list[Answer]          # Answer(question_id, value)
    resolved_slots: list[Slot]     # source=USER
    unanswered: list[str]          # id вопросов без ответа
    usage: Usage
```

3. Что я делаю с твоим ответом

Твой ответ	Действие оркестратора
`ASK`	сохраняю вопросы, показываю их пользователю (один — как есть, несколько — нумерованным списком), задача → `waiting_user`, `clarification_rounds += 1`
`PROCEED`	слоты и допущения → `task.frame`, задача → `executing`. LLM видит слоты с источниками; допущения я сам добавляю в финальный ответ блоком «Допущения»
`ABORT`	задача → `failed`, пользователь видит твой `reason`
Исключение	задача → `failed(internal_error)`; ретраи LLM и разбор JSON делай внутри движка
`parse_answers` → `ANSWER` / `UNCLEAR`	сохраняю ответы, слоты ставлю с `source=USER`, вопросы без ответа → `skipped`, снова вызываю `assess(after_answer)`
`parse_answers` → `CANCEL`	задача → `cancelled`
`parse_answers` → `NEW_REQUEST`	старая задача → `cancelled`, сообщение становится новой задачей

4. Правила, которые я проверяю

После каждого `assess` оркестратор проверяет ответ и при нарушении исправляет его сам, с записью `contract_violation` в лог. Лучше, чтобы исправлять было нечего.

1. `ASK` → от 1 до `max_questions` вопросов, по убыванию `priority`, не больше одного вопроса на слот. Лишние я отброшу.
2. Не спрашивать про слот, который уже есть с `source=USER`.

3. `rounds_left == 0` → не `ASK`. Есть открытый блокирующий пробел → `ABORT`, иначе `PROCEED`.
4. `PROCEED` только если не осталось открытых блокирующих пробелов. Открытый = `blocking=True` и в frame нет слота с источником USER, TOOL или CONTEXT.
5. Допущение никогда не закрывает блокирующий пробел — это главное правило MVP («не домысливать критичное»).
6. `resolved_slots` из `assess` — только `source=CONTEXT`; из `parse_answers` — только `source=USER`.
7. `parse_answers`: `answers` только на вопросы из `pending`; `ANSWER` без единого ответа = `UNCLEAR`; у `CANCEL` и `NEW_REQUEST` ответов нет.

Слоты: имена выбираешь сам, кроме зарезервированного `project_id` (значение = `Project.id`): по нему оркестратор выбирает проект для инструментов и контекста.

5. LLM внутри движка

Свой клиент LLM ты получаешь в конструкторе — тот же, что у оркестратора:

```
class LLMClient(Protocol):
    async def complete(self, messages: list[LLMMessage], tools:
list[ToolSpec], *, max_tokens: int = 4096) -> LLMResponse
# LLMResponse: text | None; tool_calls: list[ToolCall]; usage: Usage
# ошибки: LLMTransientError (можно повторить), LLMFatalError (нельзя)
```

Для структурированного вывода удобно передать один «инструмент» со схемой `Assessment` и читать его аргументы. Токены возвращай в `usage` — они считаются в бюджет задачи. Желательно укладывать `assess` в 10 с: пользователь ждёт либо вопрос, либо начало работы.

6. Пример: «Проверь тесты проекта», в чате два проекта

Ход 1, `assess(trigger=initial, round=1, rounds_left=3)` →

```
{
  "decision": "ask",
  "goal": "Проверить тесты проекта",
  "gaps": [
    {"id": "gap_1", "slot_name": "project_id", "description": "не определён
проект", "blocking": true,
     "information_gain": 1.0, "impact": 1.0, "user_cost": 0.1,
"available_in_context": 0.0},
    {"id": "gap_2", "slot_name": "test_scope", "description": "какие тесты
запускать", "blocking": false,
     "information_gain": 0.2, "impact": 0.3, "user_cost": 0.4,
"available_in_context": 0.0}
  ],
  "questions": [
    {"gap_id": "gap_1", "slot_name": "project_id", "text": "Какой проект
проверить?",
     "options": ["backend", "frontend"], "priority": 5.9}
  ],
  "reason": "в чате два проекта, из контекста не понятно какой"
}
```


Пользователь отвечает «backend» → `parse_answers` → `{"intent": "answer", "answers": [{"question_id": "q_1", "value": "p_backend"}], "resolved_slots": [{"name": "project_id", "value": "p_backend", "source": "user"}]}`.

Ход 2, `assess(trigger=after_answer, round=2, rounds_left=2)` → `{"decision": "proceed", "assumptions": [{"slot_name": "test_scope", "value": "all", "rationale": "не указано, какие тесты — проверяю все"}]}`.

7. Что решаешь сам

Как находить пробелы (правила, промпты, их сочетание); формула `priority = 3·blocking + 2·information_gain + impact − user_cost − available_in_context` и её веса; формулировки и варианты ответов; как понимать свободный ответ и распознавать отмену или новый запрос.

8. Вопросы к тебе

- [ ] Нужен ли тебе список инструментов агента (`tools: list[ToolSpec]`) в `AssessmentInput` ? С ним можно оценить `available_in_context` : не спрашивать то, что агент найдёт сам (команду запуска тестов в README).
- [ ] Хватает ли `history` только текущей задачи, или нужны последние сообщения чата целиком? Сейчас они приходят через `context` .

- [ ] Новый запрос вместо ответа: отменять старую задачу или сначала спрашивать? Сейчас отменяем.
- [ ] Вернуть вопросы с вариантами и свободным ответом в одном раунде — как ты будешь разбирать один текст ответа на несколько вопросов?
- [ ] Когда будет первая версия, хотя бы на правилах?